

Nom:

Prénom:

Note:

## Exercice 1 ►

/4 points

1. Ecrire un programme python qui vérifie si une liste  $L$  de longueur  $n$  est triée dans l'ordre croissant. Renvoie True si la liste est triée et False sinon. On utilisera obligatoirement une boucle.

```
def est_triee(L:list)->bool:
    ''' Renvoie True si L est triée en ordre croissant et False sinon'''
    n = len(L)
    for i in range(0, n-1):
        if L[i] > L[i+1]:
            return False
    return True

assert est_triee([1, 2, 3, 4, 5]) == True
assert est_triee([1, 2, 8, 4, 5, 6]) == False
```

2. Donner un exemple de liste de 4 éléments pour lequel `est_triee` utilise le plus de boucles.

[1, 2, 3, 4]

3. Donner un exemple de liste de 4 éléments pour lequel `est_triee` utilise le moins de boucles.

2, 1, 3, 4

## Exercice 2 ►

/4 points

1. Exécuter cet algorithme pour les mots `arbre` et `radar`. Quelle est la fonction de cet algorithme?

Cet algorithme permet de détecter si `mot` est un palindrome (s'il peut s'écrire de la même manière dans un sens ou dans l'autre)

3. Montrer que cet algorithme se termine

$j$  diminue de 1 et  $i$  augmente de 1 à chaque tour de boucle. Le variant de boucle diminue donc de 2 à chaque tour. Or l'arrêt s'effectue si ce variant est négatif. L'algorithme se termine donc.

2. Déterminer le variant de boucle de cet algorithme

 $j - i + 1$ 

## Exercice 3 ►

/3 points

1. Effectuer la trace de cet algorithme pour  $n = 4$ .

| p debut boucle | i debut boucle | while | p fin boucle | i fin de boucle |
|----------------|----------------|-------|--------------|-----------------|
| 1              | 0              | True  | 2            | 1               |
| 2              | 1              | True  | 4            | 2               |
| 4              | 2              | True  | 8            | 3               |
| 8              | 3              | True  | 16           | 4               |
| 16             | 4              | False |              |                 |
|                |                |       |              |                 |

2. Proposer un invariant de boucle et démontrer que cet algorithme est correct.

L'invariant de boucle est  $2^i = p$

- Initialisation: Avant le premier tour de boucle,  $i = 0$  et  $p = 1 = 2^0$ .
- Maintien: On suppose que l'invariant est vrai avant le tour de boucle. A la fin du tour de boucle, on a  $p = 2^i * 2 = 2^{i+1}$  et  $i = i + 1$ . Donc l'invariant est maintenu.
- Arrêt: Quand la boucle s'arrête,  $i = n$  et donc  $p = 2^n$ .

## Exercice 4 ►

/5 points

On considère le programme en pseudo code suivant qui permet de réaliser un tri par insertion en place dans une liste.

Le premier élément de la liste est trié

pour tous les éléments de la liste non triée:

    k est l'indice de la clé (premier élément de la liste non triée)

    Tant que le bord gauche n'est pas atteint et que la clé est plus petite que l'élément à sa gauche:

        échanger la clé avec l'élément à sa gauche

        l'indice de la clé diminue de 1

1. Traduire en langage python cet algorithme.

```
def tri_insertion(liste):
    ''' Tri la liste par insertion - Ne renvoie rien '''
    n = len(liste)
    for i in range(1, n):
        # i est l'indice du premier élément non trié
        k = i # k est l'indice de la clé
        # tant que la clé est plus petite que l'élément à sa gauche
        while k > 0 and liste[k] < liste[k-1]:
            # échanger la clé avec l'élément à sa gauche
            liste[k], liste[k-1] = liste[k-1], liste[k]
            k = k - 1
            # l'indice de la clé diminue de 1
```

3. Montrer que cet algorithme se termine.

2. Dans le tableau ci-dessous montrer le contenu de la liste [7, 2, 9, 1, 5, 3, 8, 4, 6] au fur et à mesure que se déroule le tri par insertion.

| Indice: | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---------|---|---|---|---|---|---|---|---|---|
| Départ  | 7 | 2 | 9 | 1 | 5 | 3 | 8 | 4 | 6 |
| Etape 1 | 2 | 7 | 9 | 1 | 5 | 3 | 8 | 4 | 6 |
| Etape 2 | 2 | 7 | 9 | 1 | 5 | 3 | 8 | 4 | 6 |
| Etape 3 | 1 | 2 | 7 | 9 | 5 | 3 | 8 | 4 | 6 |
| Etape 4 | 1 | 2 | 5 | 7 | 9 | 3 | 8 | 4 | 6 |
| Etape 5 | 1 | 2 | 3 | 5 | 7 | 9 | 8 | 4 | 6 |
| Etape 6 | 1 | 2 | 3 | 5 | 7 | 8 | 9 | 4 | 6 |
| Etape 7 | 1 | 2 | 3 | 4 | 5 | 7 | 8 | 9 | 6 |
| Etape 8 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

la boucle for se termine car elle est limitée par la taille de la liste. La boucle while se termine aussi car l'indice de la clé diminue de 1 à chaque tour et ne peut pas être négatif. De plus, la clé ne peut pas être plus petite que l'élément à sa gauche indéfiniment. Donc la boucle while se termine aussi.